Arquitectura del Sistema de Gestión de Inventario Institucional

---

Tabla de Contenidos

1. Descripción General
2. Arquitectura Seleccionada: Por Capas (Layered Architecture)
    - 2.1 ¿Por qué NO Microservicios?
    - 2.2 ¿Por qué NO MVC puro?
    - 2.3 ¿Por qué SÍ Arquitectura por Capas?
3. Vista de Capas del Sistema
4. Vista de Componentes y Despliegue
5. Flujo de una Petición Típica
6. Diagrama de Base de Datos (ER)
    - 6.1 Descripción de las Tablas
    - 6.2 Tipos y Enumeraciones
    - 6.3 Relaciones
7. Stack Tecnológico

---

1. Descripción General

El sistema es una aplicación de **gestión de inventario institucional** compuesta por tres componentes principales que se comunican entre sí:

Componente	Tecnología	Rol
Cliente	Flutter (Dart)	Interfaz de usuario multiplataforma
Backend	Node.js + Express	API REST — lógica de negocio
Base de datos	PostgreSQL 15	Persistencia de datos
Infraestructura	Docker / Docker Compose	Orquestación de contenedores

---

2. Arquitectura Seleccionada: Por Capas (Layered Architecture)

> **Veredicto: Arquitectura por Capas (N-Tier / Layered Architecture)** con patrones de Repositorio en el backend y presentación reactiva en el cliente.

2.1 ¿Por qué NO Microservicios?

Criterio	Microservicios	Este proyecto
Número de servicios backend	Múltiples procesos independientes	**Un solo proceso Express**
Bases de datos	Cada servicio tiene la suya	**Una sola base de datos PostgreSQL**
Comunicación	Event bus / gRPC / HTTP entre servicios	Llamadas directas al pool SQL dentro del mismo proceso
Despliegue	Orquestador (K8s) complejo	**Docker Compose con 2 contenedores**
Equipo requerido	Equipos grandes distribuidos	Proyecto académico / PYME

Los módulos (`auth` , `users` , `assets` , `inventory` , `maintenance` , `disposal`) son **rutas Express dentro del mismo proceso**, no servicios independientes. Comparten la misma conexión a la base de datos (`db.js`). Esto descarta por definición la arquitectura de microservicios.

2.2 ¿Por qué NO MVC puro?

El patrón MVC clásico asume que el servidor renderiza vistas (HTML). En este proyecto:

- **No hay vistas en el servidor** — el backend solo devuelve JSON.
- La "vista" vive completamente en **Flutter** (cliente desacoplado).
- El backend no tiene capa de plantillas (no hay Handlebars, EJS, etc.).

Aunque conceptualmente los **routes** de Express cumplen el rol de Controlador y las tablas PostgreSQL cumplen el rol de Modelo, el MVC clásico no aplica como patrón completo dado que el View reside en un proceso y plataforma completamente diferente.

2.3 ¿Por qué SÍ Arquitectura por Capas?

El proyecto exhibe una separación limpia e inequívoca en capas:

```
┌──────────────────────────────────────────────────┐
│         CAPA DE PRESENTACIÓN (Flutter)           │
│   Widgets • AppState • HTTP Client (dart:http)   │
├──────────────────────────────────────────────────┤
│         CAPA DE API / CONTROLADORES (Express)    │
│   Routes: auth · users · assets · inventory      │
│           maintenance · disposal                 │
│   Middlewares: helmet · cors · json · error handler │
├──────────────────────────────────────────────────┤
│         CAPA DE ACCESO A DATOS (db.js)           │
│   PostgreSQL Pool (pg) — consultas parametrizadas │
├──────────────────────────────────────────────────┤
│         CAPA DE DATOS (PostgreSQL 15)            │
│   Tablas · ENUMs · Triggers · Transacciones      │
└──────────────────────────────────────────────────┘
```

Justificaciones concretas:

1. **Separación de responsabilidades verificable en el código:**
    - `db.js` → única responsabilidad: gestionar el pool de conexiones.
    - Cada archivo en `routes/` → única responsabilidad: manejar un dominio de negocio.
    - `index.js` → única responsabilidad: ensamblar middlewares y registrar rutas.
    - `main.dart` → presenta la UI y delega lógica al `AppState`.
2. **Comunicación estrictamente vertical:** Flutter llama al backend vía HTTP REST; el backend llama a PostgreSQL vía el pool SQL. Ninguna capa se salta a otra.
3. **Independencia de despliegue por capa:** Flutter puede correr en Android, iOS o Web sin cambiar el backend. El backend puede escalar sin tocar la UI. La base de datos puede migrarse sin modificar el código de la API (solo las queries).
4. **Adecuación al tamaño del proyecto:** La arquitectura por capas es la más apropiada para un proyecto académico / institucional de este tamaño porque minimiza la complejidad operacional (solo 2 contenedores Docker) sin sacrificar mantenibilidad.

---

3. Vista de Capas del Sistema

```
graph TB
    subgraph CLIENTE["🖥️  CAPA DE PRESENTACIÓN — Flutter (Dart)"]
        UI["Widgets / Páginas<br/>(LoginPage · HomePage · AssetDetailPage ...)"]
        STATE["AppState (ChangeNotifier)<br/>Gestión de estado reactivo"]
        HTTP["HTTP Client<br/>dart:http → REST API"]
        LOCAL["SharedPreferences<br/>Persistencia local de sesión"]
    end

    subgraph BACKEND["⚙️  CAPA DE API — Node.js + Express"]
        MW["Middlewares<br/>helmet · cors · express.json"]
        ROUTES["Rutas / Controladores"]
        AUTH_R["/api/auth"]
        USERS_R["/api/users"]
        ASSETS_R["/api/assets"]
        INV_R["/api/inventory"]
        MAINT_R["/api/maintenance"]
        DISP_R["/api/disposal"]
        ERR["Error Handler Global"]
    end

    subgraph DAL["🔌  CAPA DE ACCESO A DATOS — db.js"]
        POOL["PostgreSQL Connection Pool<br/>(node-postgres / pg)"]
    end

    subgraph DB["🗄️  CAPA DE DATOS — PostgreSQL 15"]
        TABLES["Tablas, ENUMs, Triggers"]
    end

    UI --> STATE
    STATE --> HTTP
    HTTP -- "HTTP/REST JSON" --> MW
    MW --> ROUTES
    ROUTES --> AUTH_R & USERS_R & ASSETS_R & INV_R & MAINT_R & DISP_R
    AUTH_R & USERS_R & ASSETS_R & INV_R & MAINT_R & DISP_R --> POOL
    AUTH_R & USERS_R & ASSETS_R & INV_R & MAINT_R & DISP_R --> ERR
    POOL -- "SQL parametrizado" --> TABLES
```

---

4. Vista de Componentes y Despliegue

```
graph LR
    subgraph DEVICE["Dispositivo / Navegador"]
        FLUTTER["App Flutter\n(Android · iOS · Web · Desktop)"]
    end

    subgraph DOCKER["Docker Compose Host"]
        subgraph C1["Contenedor: inventario_backend\npuerto 3000"]
            EXPRESS["Express API\nNode.js 18"]
        end
        subgraph C2["Contenedor: inventario_db\npuerto 5432"]
            PG["PostgreSQL 15"]
            VOL[("Volumen\ninventario_data")]
        end
        INIT["init.sql\n(seed inicial)"]
    end

    FLUTTER -- "HTTP :3000" --> EXPRESS
    EXPRESS -- "TCP :5432" --> PG
    PG --- VOL
    INIT -.->|"docker-entrypoint-initdb"| PG
```

---

5. Flujo de una Petición Típica

> Ejemplo: el auxiliar de inventario registra un activo nuevo.

```
sequenceDiagram
    actor Usuario
    participant Flutter as Flutter (UI + AppState)
    participant Express as Express /api/assets (POST)
    participant DB as PostgreSQL

    Usuario->>Flutter: Rellena formulario y pulsa "Guardar"
    Flutter->>Express: POST /api/assets {code, name, category, ...}
    Express->>Express: Valida campos requeridos
    Express->>DB: INSERT INTO assets (...) RETURNING *
    DB-->>Express: Fila insertada
    DB->>DB: Trigger assets_updated_at actualiza updated_at
    Express->>DB: INSERT INTO asset_history (acción "CREADO")
    DB-->>Express: OK
    Express-->>Flutter: 201 Created {asset JSON}
    Flutter->>Flutter: AppState actualiza lista local
    Flutter-->>Usuario: Muestra confirmación en pantalla
```

---

6. Diagrama de Base de Datos (ER)

```
erDiagram

    users {
        VARCHAR_20  id            PK
        VARCHAR_100 username      UK
        VARCHAR_200 full_name
        VARCHAR_200 email         UK
        VARCHAR_255 password_hash
        user_role[] roles
        VARCHAR_200 area
        BOOLEAN     is_active
        TIMESTAMPTZ last_session
        INT         failed_attempts
        TIMESTAMPTZ lock_until
        TIMESTAMPTZ created_at
    }

    assets {
        VARCHAR_50  code                       PK
        VARCHAR_200 name
        VARCHAR_100 category
        VARCHAR_100 subcategory
        VARCHAR_200 physical_location
        VARCHAR_200 responsible
        VARCHAR_200 dependency
        VARCHAR_100 cost_center
        NUMERIC     acquisition_value
        DATE        acquisition_date
        INT         estimated_useful_life_years
        asset_state state
        TEXT        observations
        VARCHAR_200 program
        VARCHAR_500 photo_path
        TIMESTAMPTZ created_at
        TIMESTAMPTZ updated_at
    }

    asset_history {
        SERIAL      id           PK
        VARCHAR_50  asset_code   FK
        TIMESTAMPTZ timestamp
        VARCHAR_100 action
        TEXT        detail
        VARCHAR_200 performed_by
    }

    inventory_sessions {
        VARCHAR_50  id         PK
        VARCHAR_200 name
        VARCHAR_200 site
        VARCHAR_200 building
        VARCHAR_100 floor
        VARCHAR_200 area
        TIMESTAMPTZ created_at
    }

    inventory_session_baseline {
        VARCHAR_50  session_id     FK "PK compuesta"
        VARCHAR_50  asset_code     FK "PK compuesta"
        asset_state baseline_state
    }

    inventory_verifications {
        SERIAL              id          PK
        VARCHAR_50          session_id  FK
        VARCHAR_50          asset_code
        verification_result result
        TEXT                notes
        TIMESTAMPTZ         timestamp
        VARCHAR_500         photo_path
    }

    maintenance_requests {
        VARCHAR_50       id          PK
        VARCHAR_50       asset_code  FK
```

```
        maintenance_type type
        TEXT             description
        VARCHAR_200      created_by
        TIMESTAMPTZ      created_at
        BOOLEAN          closed
    }

    disposal_requests {
        VARCHAR_50  id                     PK
        VARCHAR_50  asset_code             FK
        VARCHAR_200 cause
        TEXT        justification
        VARCHAR_200 created_by
        TIMESTAMPTZ created_at
        BOOLEAN     approved_by_dependency
        BOOLEAN     approved_by_daf
    }

    %% ── Relaciones ──────────────────────────────────────────────
    assets                    ||--o{ asset_history              : "registra historial de"
    assets                    ||--o{ inventory_session_baseline : "se incluye en"
    inventory_sessions        ||--o{ inventory_session_baseline : "define baseline de"
    inventory_sessions        ||--o{ inventory_verifications    : "contiene"
    assets                    ||--o{ maintenance_requests       : "genera solicitud de"
    assets                    ||--o{ disposal_requests          : "puede tener"
```

6.1 Descripción de las Tablas

Tabla	Dominio	Descripción
`users`	Seguridad	Usuarios del sistema con roles RBAC, control de bloqueo por intentos fallidos y auditoría de última sesión.
`assets`	Inventario	Catálogo maestro de activos fijos institucionales. Es la tabla central referenciada por todas las demás.
`asset_history`	Trazabilidad	Log inmutable de cambios realizados a cada activo (quién, cuándo, qué acción).
`inventory_sessions`	Verificación	Sesiones de toma física de inventario, asociadas a un sitio, edificio y área específicos.
`inventory_session_baseline`	Verificación	Snapshot del estado esperado de cada activo al inicio de una sesión (tabla puente N:M).
`inventory_verifications`	Verificación	Registros del resultado real de cada activo verificado durante una sesión (incluyendo foto y notas).
`maintenance_requests`	Mantenimiento	Solicitudes de mantenimiento preventivo o correctivo asociadas a un activo.
`disposal_requests`	Bajas	Solicitudes de baja o disposición final de activos, con flujo de aprobación por dependencia y DAF.

6.2 Tipos y Enumeraciones

Tipo ENUM	Valores
`user_role`	`auxiliarInventario`, `administrador`, `responsableArea`, `direccionAdminFin`, `auditor`, `soporteTI`
`asset_state`	`activo`, `reubicado`, `noEncontrado`, `obsoleto`, `enReparacion`, `paraBaja`
`verification_result`	`encontrado`, `reubicado`, `noEncontrado`, `paraBaja`, `obsoleto`, `enReparacion`
`maintenance_type`	`preventivo`, `correctivo`

6.3 Relaciones

Relación	Cardinalidad	Regla de integridad
`assets` → `asset_history`	1 : N	`ON DELETE CASCADE` — si se elimina un activo, se elimina todo su historial.
`assets` → `inventory_session_baseline`	1 : N	`ON DELETE CASCADE` — clave compuesta `(session_id, asset_code)`.
`inventory_sessions` → `inventory_session_baseline`	1 : N	`ON DELETE CASCADE` — tabla puente de la relación N:M sesión/activo.
`inventory_sessions` → `inventory_verifications`	1 : N	`ON DELETE CASCADE` — cada verificación pertenece a una sesión.
`assets` → `maintenance_requests`	1 : N	`ON DELETE CASCADE` — un activo puede tener múltiples solicitudes de mantenimiento.
`assets` → `disposal_requests`	1 : N	`ON DELETE CASCADE` — un activo puede tener múltiples solicitudes de baja.

> **Nota sobre `inventory_verifications.asset_code`**: este campo no tiene FK declarada en la base de datos (flexible por diseño), lo que permite registrar verificaciones de activos escaneados que aún no estén en el catálogo.

---

7. Stack Tecnológico

```
graph LR
    subgraph FRONTEND
        F1["Flutter 3.x (Dart)"]
        F2["material3 · google_fonts"]
        F3["mobile_scanner (QR/código de barras)"]
        F4["image_picker · pdf (generación de reportes)"]
        F5["shared_preferences (sesión local)"]
        F6["http (REST client)"]
    end

    subgraph BACKEND
        B1["Node.js 18 + Express 4"]
        B2["helmet (HTTP security headers)"]
        B3["cors (Cross-Origin)"]
        B4["pg / node-postgres (pool SQL)"]
    end

    subgraph DATABASE
        D1["PostgreSQL 15-alpine"]
        D2["pgcrypto (bcrypt passwords)"]
        D3["Trigger set_updated_at"]
        D4["ENUM types · Arrays"]
    end

    subgraph INFRA
        I1["Docker Compose 3.9"]
        I2["Health check (pg_isready)"]
        I3["Volumen persistente inventario_data"]
    end

    FRONTEND -- "HTTP REST / JSON" --> BACKEND
    BACKEND -- "TCP / SQL" --> DATABASE
    INFRA -. "orquesta" .- BACKEND
    INFRA -. "orquesta" .- DATABASE
```

Documento generado a partir del análisis del código fuente del repositorio. Última actualización: Abril 2026